LAPORAN FINAL PROJECT

TRACKER BIPOL

Oleh Kelompok :

Citra Amelia	(2303332067)
Namira Assalwa	(2407412025)
Branita Adwitiya Diwyacitta	(2303332091)
Ikhsandi Ocean	(2307422036)

DIVISI *EXPLORE*

KSM COMPUTER STUDENT CLUB

POLITEKNIK NEGERI JAKARTA

DEPOK

2025

KATA PENGANTAR

Pertama-tama, marilah kita mengucapkan rasa syukur ke hadirat Tuhan Yang Maha Esa yang telah memberikan rahmat dan bimbingannya kepada penulis. Berkat dukungan-Nya, penulis dapat menyelesaikan laporan ini. Selain itu, penulis juga memanjatkan doa dan salam kepada Rasulullah, keluarga, dan semua yang telah mendukung penulis hingga saat ini.

Penulis mengucapkan terima kasih kepada orang tua dan mentor yang telah memberikan dukungan serta kesempatan kepada penulis dalam menyelesaikan laporan ini. Laporan ini disusun sebagai bagian dari tugas akhir KSM dan merupakan tugas sebagai anggota KSM Computer Student Club. Tujuan utama dari laporan ini adalah untuk menambah pengetahuan tentang IOT, khususnya “Tracker BIPOL”.

Dalam laporan ini, penulis telah menyajikan teori dasar, analisis pro dan kontra, serta solusi terhadap masalah yang berkaitan dengan “Tracker BIPOL”. Penulisan laporan ini tidak hanya memperluas pengetahuan penulis mengenai tracker bus, tetapi juga menyadarkan pentingnya kerja sama tim dalam mencapai tujuan bersama.

Semoga laporan ini bermanfaat bagi pembaca dan dapat menjadi referensi yang berguna dalam memahami “Tracker BIPOL”.

Depok, 18 Juli 2025

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Transportasi internal kampus merupakan infrastruktur vital yang mempengaruhi aktivitas akademik seluruh civitas academica. Di banyak perguruan tinggi, armada bus kampus menjadi tulang punggung mobilitas sehari-hari, menghubungkan berbagai fasilitas seperti gedung perkuliahan, area parkir, dan lain-lain. Namun, ketiadaan sistem informasi real-time tentang posisi dan estimasi kedatangan bus sering menimbulkan masalah operasional yang signifikan. Mahasiswa kerap menghabiskan waktu menunggu yang tidak produktif di halte, dosen terkadang terlambat mengajar karena ketidakpastian jadwal, sementara pihak pengelola transportasi kesulitan memonitor dan mengoptimalkan operasional armada.

Kondisi ini semakin krusial mengingat karakteristik khusus transportasi kampus. Layanan bus biasanya beroperasi pada rute tetap dengan jarak tempuh relatif pendek, namun frekuensi penggunaannya sangat padat terutama pada jam-jam sibuk antara perkuliahan. Pola pergerakannya yang teratur dan area cakupan yang terbatas justru seharusnya memudahkan implementasi sistem pelacakan digital. Sayangnya, sebagian besar kampus masih mengandalkan sistem konvensional tanpa teknologi tracking, padahal solusi berbasis IoT sebenarnya dapat diwujudkan dengan biaya efektif menggunakan komponen seperti ESP32, ESP8266, modul GPS, dan antarmuka LCD.

Perkembangan teknologi IoT saat ini telah mencapai tingkat kematangan yang memungkinkan implementasi sistem pelacakan dengan akurasi tinggi dan biaya rendah. Modul GPS generasi terbaru seperti NEO-6M mampu memberikan data lokasi dengan ketepatan hingga 2,5 meter, sementara mikrokontroler ESP32 dan ESP8266 menawarkan kemampuan komputasi yang memadai sekaligus konektivitas WiFi terintegrasi. Kombinasi teknologi ini memungkinkan pembuatan sistem yang tidak hanya akurat tetapi juga mudah diintegrasikan dengan infrastruktur kampus yang ada. Dengan dukungan teknologi tersebut, seharusnya setiap kampus mampu mengembangkan sistem transportasi yang lebih cerdas dan responsif.

Implementasi sistem bus tracker juga sejalan dengan tren global dalam pengembangan smart campus. Banyak universitas terkemuka dunia telah mengadopsi solusi serupa untuk meningkatkan kualitas layanan transportasi kampusnya. Sistem ini tidak hanya menyelesaikan masalah operasional harian, tetapi juga berkontribusi dalam pengumpulan data untuk perencanaan transportasi jangka panjang. Data historis pergerakan bus dapat dianalisis untuk mengoptimalkan rute, menyesuaikan jumlah armada, dan meningkatkan efisiensi operasional. Dalam jangka panjang, sistem ini akan menjadi bagian penting dari ekosistem kampus yang terhubung dan berbasis data.

Pengembangan bus tracker kampus berbasis teknologi ini menawarkan solusi komprehensif bagi berbagai pihak. Bagi pengguna, sistem ini memberikan kemudahan akses informasi real-time melalui tampilan LCD di halte. Bagi pengelola transportasi, data akurat tentang posisi armada memungkinkan optimalisasi rute, penyesuaian frekuensi, dan perbaikan layanan berbasis analisis riwayat pergerakan. Dari perspektif institusi, implementasi sistem ini sekaligus menjadi wujud komitmen terhadap transformasi digital dan konsep smart campus.

Proyek ini juga memiliki nilai edukatif yang penting. Pengembangan prototipe bus tracker menjadi media pembelajaran langsung bagi mahasiswa anggota KSM Computer Student Club untuk mengaplikasikan pengetahuan tentang pemrograman IOT. Kolaborasi antara unit transportasi kampus dapat menciptakan sinergi yang bermanfaat bagi pengembangan sistem lebih lanjut. Dengan mempertimbangkan semua aspek tersebut, implementasi sistem pelacakan bus kampus berbasis ESP32, ESP8266, GPS, dan LCD layak menjadi prioritas dalam modernisasi layanan transportasi perguruan tinggi.

1.2 Rumusan Masalah

Permasalahan yang diangkat dalam proyek ini adalah:

1. Bagaimana cara membuat sistem pelacakan bus secara real-time menggunakan ESP32,ESP8266, LCD, dan modul GPS?
2. Bagaimana menampilkan data lokasi bus ke pengguna melalui LCD?
3. Apa saja kendala teknis yang mungkin muncul dalam integrasi GPS, WiFi (ESP8266/ESP32), dan LCD?

1.3 Tujuan

Adapun tujuan dari pembuatan proyek ini adalah:

1. Membangun prototipe sistem Bus Tracker berbasis IoT menggunakan ESP32/ESP8266, GPS, dan LCD.
2. Menyajikan informasi lokasi bus secara real-time kepada pengguna.
3. Menguji keandalan sistem dalam hal akurasi GPS, kestabilan koneksi WiFi, dan tampilan data di LCD.
4. Memberikan solusi hemat biaya (low-cost) untuk manajemen transportasi kampus.

1.4 Batasan Masalah

1. Sistem hanya menggunakan modul GPS dan ESP32/ESP8266 sebagai mikrokontroler utama.
2. Tampilan data lokasi diperlihatkan melalui LCD.
3. Pengujian dilakukan dalam jarak terbatas di lingkungan kampus.
4. Sistem belum mempertimbangkan konsumsi daya optimal untuk penggunaan baterai jangka panjang (jika dipasang di bus nyata).

BAB II

ISI

2.1 Waktu dan Tempat

Tempat Pelaksanaan: Politeknik Negeri Jakarta dan Universitas Indonesia

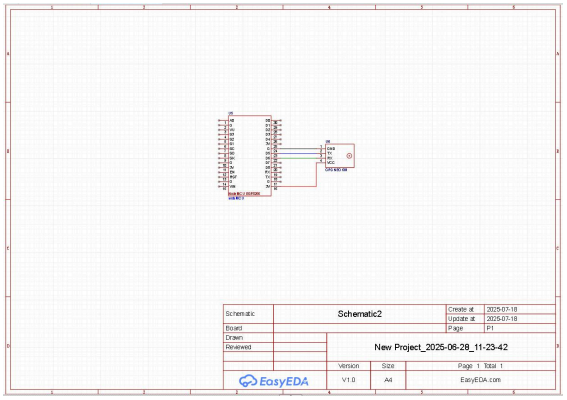
Waktu Pelaksanaan: 10 Mei 2025 s.d. 17 Juli 2025

2.2 Alat dan Bahan

Alat dan Bahan yang diperlukan untuk membuat rangkaian, sebagai berikut:

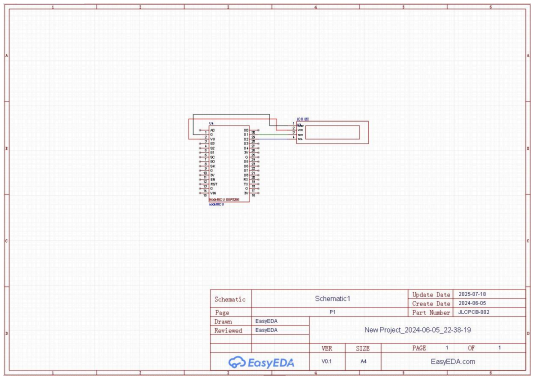
NO	NAMA ALAT DAN BAHAN	JUMLAH
1.	NodeMCU ESP8266	2
2.	GPS Neo 6M	1
3.	LCD I2C 16×2	1
4.	Baterai 18650 + Casing	2
5.	Step Up Converter	2
6.	Kabel USB to Micro	2
7.	Solder 80W	1
8.	Pasta Solder	1
9.	Timah	Secukupnya
10.	Kabel Jumper Female to Female	Secukupnya

2.3 Rancangan



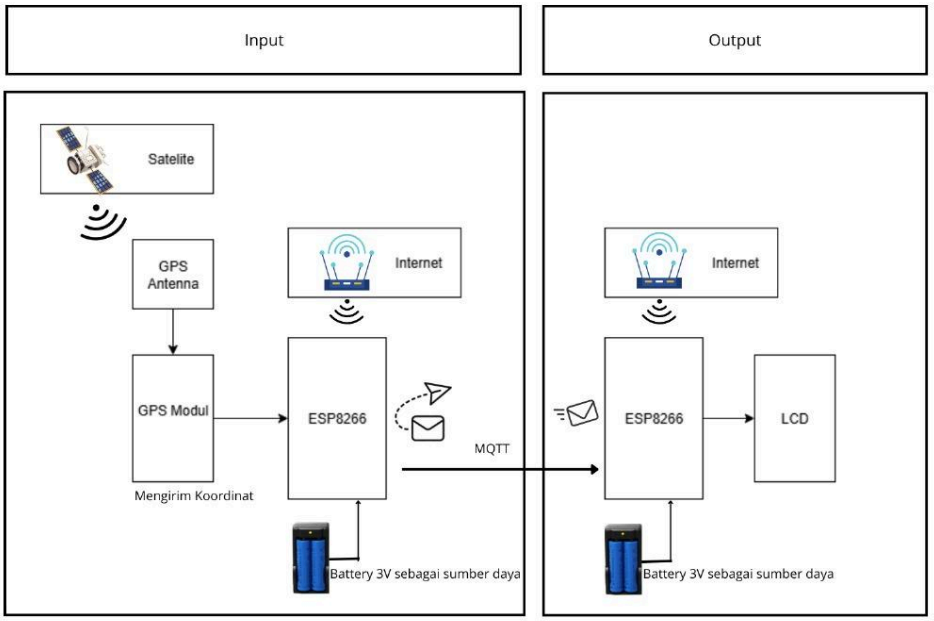
Gambar 1. Skematik Rangkaian BIPOL

- GPS module dihubungkan ke ESP8266 melalui komunikasi serial. Pin TX dari GPS disambungkan ke pin D5 pada ESP8266, dan pin RX dari GPS ke pin D6 pada ESP8266 untuk memungkinkan pengiriman dan penerimaan data koordinat. Modul ini membutuhkan daya 3.3–5V, jadi pin VCC GPS dihubungkan ke pin 3V3 dan GND GPS ke GND umum.
- ESP8266 menerima data dari GPS melalui pin RX dan TX, dan terhubung ke jaringan WiFi menggunakan kode pemrograman internal. VCC ESP8266 disuplai langsung dari baterai 18650 melalui pin VIN, dan GND ke negatif baterai. Modul ini mem-publish data lokasi ke broker MQTT publik seperti broker.hivemq.com.
- Baterai 18650 positif baterai langsung ke pin VIN ESP8266 untuk menyuplai daya 3.7V (ESP8266 masih bisa hidup di tegangan ini meskipun tidak ideal), dan negatif baterai ke GND ESP8266. Baterai ini juga menyuplai tegangan ke modul GPS melalui jalur 3.3V di ESP8266, karena pin 3.3V pada board mendapatkan daya dari regulator internal ESP8266.

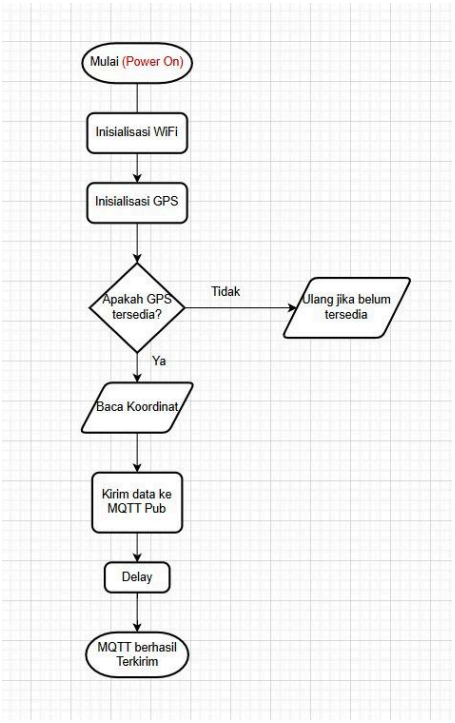


Gambar 2. Skematik Rangkaian Halte

- ESP8266 dihubungkan ke LCD I2C melalui pin D2 (SDA) dan D1 (SCL) untuk komunikasi data tampilan. ESP8266 menerima data lokasi dari broker MQTT dengan cara subscribe ke topik yang sama seperti yang digunakan oleh pengirim. Power ESP8266 disuplai melalui port micro USB menggunakan adaptor 5V atau charger HP, dan GND otomatis terhubung melalui ground USB.
- LCD I2C, pada SDA dari LCD ke D2 ESP8266 dan SCL ke D1 ESP8266 untuk komunikasi I2C, VCC ke VU, dan GND ke GND ESP8266. Modul ini digunakan untuk menampilkan data koordinat lokasi halte secara langsung.
- Adaptor USB 5V. Kabel micro USB disambungkan langsung ke port USB pada board ESP8266 untuk menyuplai daya 5V secara langsung. Sumber daya ini digunakan untuk menghidupkan ESP8266 dan LCD I2C secara stabil.



Gambar 3. Diagram Blok Tracker BIPOL



Gambar 4. Flowchart Rangkaian BIPOL



Gambar 5. Flowchart Rangkaian Halte

2.4 Langkah Kerja

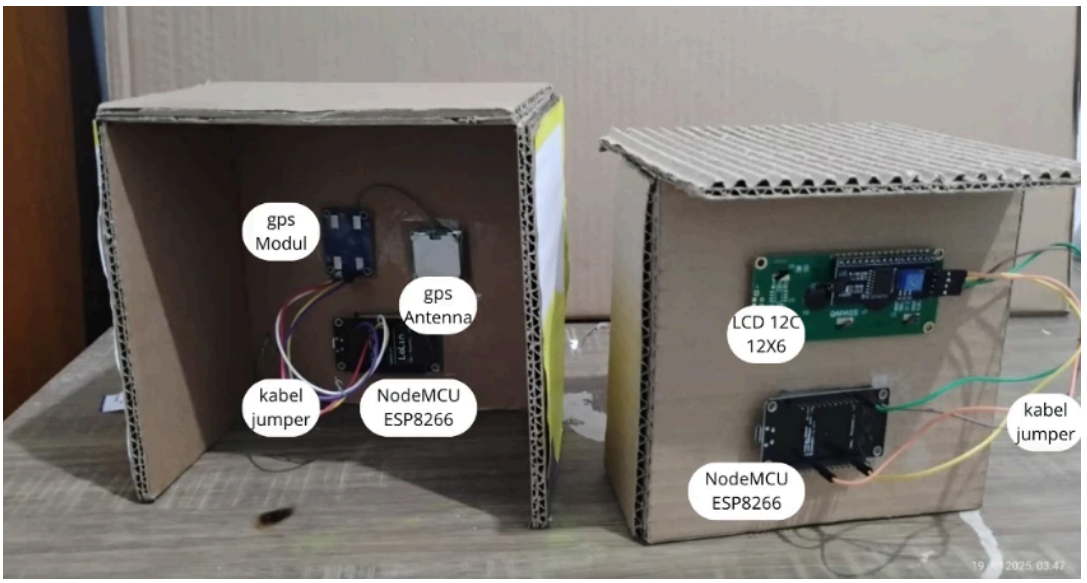
Sistem ini terdiri dari dua rangkaian utama yang saling terhubung melalui jaringan WiFi dan berkomunikasi menggunakan protokol MQTT. Rangkaian pertama berfungsi sebagai pengirim informasi lokasi. Pada rangkaian ini, modul GPS digunakan untuk mendeteksi titik koordinat halte secara real-time. Data lokasi yang diperoleh kemudian diproses oleh mikrokontroler ESP8266 yang ditenagai oleh baterai 18650 dan step-up converter 5V agar cukup untuk menyalakan modul. Setelah itu, ESP8266 terhubung ke jaringan WiFi dan mengirimkan data koordinat ke broker MQTT publik dengan menggunakan metode publish pada topik tertentu, misalnya “Halte Vokasi”.

Sementara itu, rangkaian kedua berperan sebagai penerima informasi. Rangkaian ini terdiri dari ESP8266 yang mendapatkan daya langsung dari adaptor atau kabel USB tanpa menggunakan baterai. ESP8266 pada rangkaian ini juga terhubung ke jaringan WiFi yang sama atau serupa, kemudian melakukan subscribe ke topik MQTT yang digunakan oleh pengirim. Ketika data baru dipublikasikan oleh rangkaian pertama, ESP8266 di rangkaian kedua akan menerima pesan tersebut secara otomatis. Informasi koordinat yang diterima kemudian ditampilkan pada layar LCD I2C, sehingga pengguna dapat mengetahui lokasi halte bus secara langsung.

2.5 Hasil



Gambar 6. Hasil Project Tampak Depan



Gambar 7. Hasil Project Tampak Belakang

Proyek tracer bus ini berhasil membangun sistem pelacakan lokasi secara real-time menggunakan dua rangkaian utama yang saling terhubung melalui jaringan internet, dengan inti pemrosesan pada mikrokontroler ESP8266. Rangkaian pertama berfungsi sebagai pengirim data dan dipasang di lokasi halte atau di dalam kendaraan yang bergerak, terdiri dari modul GPS (misalnya NEO-6M) dan board ESP8266. Modul GPS bertugas menerima sinyal dari beberapa satelit di orbit bumi, dan dari sinyal-sinyal tersebut dihitung posisi koordinat saat ini berupa garis lintang (latitude) dan garis bujur (longitude). Koordinat ini dikirim secara serial melalui pin TX dari GPS ke pin RX di ESP8266. Setelah data diterima oleh ESP8266, koordinat tersebut akan diolah dari format aslinya (biasanya format NMEA) menjadi format teks biasa yang lebih mudah dibaca dan dikirimkan melalui internet menggunakan protokol komunikasi MQTT. ESP8266 melakukan proses publish, yaitu mengirimkan data ke topik tertentu pada broker MQTT publik, misalnya ke topik "/tracerbus/koordinat".

Sementara itu, rangkaian kedua berfungsi sebagai penerima dan penampil data. Di dalamnya, terdapat board ESP8266 lain yang terhubung ke Wi-Fi dan berlangganan (subscribe) ke topik MQTT yang sama. Ketika ada data koordinat yang baru dikirimkan oleh rangkaian pertama, maka data tersebut akan langsung diterima oleh rangkaian kedua secara otomatis. Koordinat yang diterima akan ditampilkan ke LCD I2C, yaitu layar teks digital yang terhubung ke ESP8266 melalui dua jalur komunikasi I2C (SDA dan SCL). Layar ini akan menampilkan informasi lokasi seperti "Lat: -6.123456" dan "Long:

106.987654", atau bisa juga disesuaikan untuk menunjukkan informasi teks seperti “Bus tiba di halte A”. Komunikasi MQTT menjadikan proses pengiriman data menjadi ringan dan efisien, karena hanya menggunakan sedikit bandwidth dan bekerja cepat selama koneksi internet tersedia.

Seluruh sistem bekerja otomatis tanpa perlu dioperasikan langsung oleh pengguna. Daya untuk kedua rangkaian diperoleh dari adaptor USB 5V yang langsung disambungkan ke port micro USB pada board ESP8266, sehingga tidak membutuhkan baterai ataupun komponen konversi tegangan seperti step-up converter. Hal ini membuat perangkat lebih praktis, mudah dipasang, dan cocok digunakan dalam lingkungan indoor maupun semi-permanen di halte. Secara keseluruhan, sistem ini telah diuji berjalan dengan cukup stabil, dapat mengirim dan menampilkan koordinat lokasi dengan lancar, asalkan sinyal GPS dan koneksi internet tersedia. Proyek ini membuktikan bahwa dengan perangkat yang sederhana dan biaya terjangkau, sistem pelacakan kendaraan atau posisi halte dapat dibangun secara mandiri dan efektif, serta menjadi dasar yang baik untuk dikembangkan ke sistem transportasi pintar di masa depan.

2.6 Evaluasi Permasalahan

Selama pengujian sistem, salah satu permasalahan yang terjadi adalah ketidaksesuaian antara titik koordinat yang ditampilkan di LCD dengan posisi sebenarnya saat dilihat di Google Maps. Hal ini dapat disebabkan oleh beberapa faktor teknis. Pertama, modul GPS membutuhkan waktu untuk mendapatkan posisi “fix” yang akurat setelah dinyalakan, dan jika data dikirim terlalu cepat sebelum modul benar-benar fix, maka koordinat yang dikirim adalah data awal yang tidak valid. Kedua, modul GPS sangat bergantung pada kondisi lingkungan. Lokasi dengan banyak bangunan tinggi, pepohonan lebat, atau berada di dalam ruangan dapat mengganggu penerimaan sinyal satelit, sehingga koordinat yang dihasilkan kurang akurat atau bahkan salah. Ketiga, kesalahan bisa berasal dari proses parsing data NMEA (protokol data GPS) yang dilakukan oleh ESP8266. Jika parsing tidak memvalidasi format data secara benar, maka informasi lokasi yang salah bisa dikirim ke MQTT dan ditampilkan di layar. Keempat, jeda waktu pengiriman data yang terlalu cepat tanpa filter atau validasi bisa membuat sistem menampilkan data yang terus berubah meskipun posisi tidak berpindah, yang berpotensi menyesatkan pengguna. Untuk mengatasi permasalahan ini, beberapa solusi dapat diterapkan. Di antaranya adalah menambahkan logika validasi checksum dari data NMEA sebelum diproses lebih lanjut,

memastikan koordinat memiliki status fix yang valid, dan menunda pengiriman data hingga GPS benar-benar memperoleh posisi stabil. Selain itu, perlu diterapkan logika threshold atau filter pergerakan di dalam kode ESP8266 agar hanya perubahan koordinat signifikan yang dikirim. Dengan penempatan modul GPS di tempat terbuka dan penggunaan antena eksternal, kualitas sinyal juga bisa ditingkatkan untuk menghindari gangguan lingkungan.

2.7 Source Code

A. Code Rangkaian Bus

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <SoftwareSerial.h>
#include <TinyGPS++.h>

#define RX 14
#define TX 12
#define GPS_BAUD 9600

TinyGPSPlus gps;
SoftwareSerial gpsSerial(RX, TX);

const char* ssid = "hotspot1";
const char* password = "1234567899";
const char* mqtt_server = "broker.hivemq.com";

const char* mqtt_topic = "exploreCscKel3/bus";

WiFiClient espClient;
PubSubClient client(espClient);
```

```
unsigned long lastPublishTime = 0;
const long publishInterval = 5000;

float latitude;
float longitude;
String waktu = "";

void setup_wifi() {
  delay(10);
  Serial.println();
  Serial.print("Menghubungkan ke WiFi: ");
  Serial.println(ssid);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi terhubung!");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("Menghubungkan ke MQTT...");
```

```
    if (client.connect("ESP8266Client")) {
      Serial.println("terhubung!");
    } else {
      Serial.print("gagal, rc=");
      Serial.print(client.state());
      Serial.println(" coba lagi dalam 5 detik");
      delay(5000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  setup_wifi();
  client.setServer(mqtt_server, 1883);
  gpsSerial.begin(GPS_BAUD);
  Serial.println("Software Serial GPS dimulai");
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();

  // Baca data GPS terus-menerus
  while (gpsSerial.available()) {
    gps.encode(gpsSerial.read());
  }
```

```
  // Cek apakah waktunya publish
  unsigned long currentMillis = millis();
  if (currentMillis - lastPublishTime >= publishInterval) {
    lastPublishTime = currentMillis;

    // Jika ada data baru dari GPS
    if (gps.location.isValid() && gps.time.isValid()) {
      latitude = gps.location.lat();
      longitude = gps.location.lng();
      waktu = String(gps.time.hour()) + ":" +
String(gps.time.minute()) + ":" + String(gps.time.second());
    } else {
      // Default jika data tidak tersedia
      latitude = 0.0;
      longitude = 0.0;
      waktu = "00:00:00";
    }

    // Format JSON
    String jsonPayload = "{";
    jsonPayload += "\"latitude\": " + String(latitude, 6) + ",";
    jsonPayload += "\"longitude\": " + String(longitude, 6) +
",";
    jsonPayload += "\"waktu\": \"" + waktu + "\"";
    jsonPayload += "}";

    // Konversi ke buffer char*
    char buffer[256];
    jsonPayload.toCharArray(buffer, sizeof(buffer));
```

```
    // Publish
    client.publish(mqtt_topic, buffer);
    Serial.print("JSON dikirim ke topik [");
    Serial.print(mqtt_topic);
    Serial.print("]: ");
    Serial.println(jsonPayload);
  }
}
```

B. Code Rangkaian Halte

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <ArduinoJson.h>
#include <math.h>

// Ganti sesuai dengan WiFi & MQTT kamu
const char* ssid = "Redmi 13C";
const char* password = "123457890";
const char* mqtt_server = "broker.hivemq.com";
const char* mqtt_topic = "exploreCscKel3/bus";

// LCD I2C (0x27 atau 0x3F tergantung modul)
LiquidCrystal_I2C lcd(0x27, 16, 2);

WiFiClient espClient;
PubSubClient client(espClient);
```

```
// Struktur data halte
struct Halte {
  const char* nama;
  float lat;
  float lon;
};

// Daftar halte (ganti sesuai lokasi sebenarnya)
Halte daftarHalte[] = {
  { "Titik Awal Bipol", -6.3711018, 106.8237532 },
  { "Halte Vokasi", -6.366127, 106.821670 },
  { "Halte SOR", -6.366812, 106.823784 },
  { "Halte FMIPA", -6.3698232,  106.8258449 },
  { "Parkiran", -6.371597, 106.824914 },
};

// Radius toleransi (0.0005 derajat ≈ ±50 meter)
const float radiusToleransi = 0.0005

;

// Fungsi untuk deteksi halte
const char* deteksiHalte(float lat, float lon) {
  for (int i = 0; i < sizeof(daftarHalte) /
sizeof(daftarHalte[0]); i++) {
    float deltaLat = lat - daftarHalte[i].lat;
    float deltaLon = lon - daftarHalte[i].lon;
    float jarak = sqrt(deltaLat * deltaLat + deltaLon *
deltaLon);
```

```
    if (jarak <= radiusToleransi) {
      return daftarHalte[i].nama;
    }
  }
  return "Di Perjalanan";
}

void setup_wifi() {
  delay(10);
  Serial.println();
  Serial.print("Menghubungkan ke WiFi: ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi Terhubung!");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());
}

void callback(char* topic, byte* payload, unsigned int length) {
  String jsonStr = "";
  for (unsigned int i = 0; i < length; i++) {
    jsonStr += (char)payload[i];
  }

  Serial.print("Pesan diterima: ");
```

```
  Serial.println(jsonStr);

  StaticJsonDocument<200> doc;
  DeserializationError error = deserializeJson(doc, jsonStr);

  if (error) {
    Serial.print("Gagal parsing JSON: ");
    Serial.println(error.f_str());
    return;
  }

  float lat = doc["latitude"];
  float lon = doc["longitude"];
  Serial.print("Latitude: ");
  Serial.println(lat, 6);
  Serial.print("Longitude: ");
  Serial.println(lon, 6);

  const char* namaHalte = deteksiHalte(lat, lon);

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Lokasi Bus:");
  lcd.setCursor(0, 1);
  lcd.print(namaHalte);
}

void reconnect() {
  while (!client.connected()) {
```

```
    Serial.print("Menghubungkan ke MQTT...");
    if (client.connect("ESP8266DisplayClient")) {
      Serial.println("Terhubung!");
      client.subscribe(mqtt_topic);
    } else {
      Serial.print("Gagal, rc=");
      Serial.print(client.state());
      Serial.println(" coba lagi dalam 5 detik");
      delay(5000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Menunggu data...");
  setup_wifi();
  client.setServer(mqtt_server, 1883);
  client.setCallback(callback);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();
```

```
}
```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil perancangan dan pengujian, sistem tracer bus berbasis GPS dan ESP8266 ini mampu mengirimkan data koordinat halte secara real-time melalui MQTT dan menampilkannya pada LCD dengan cukup baik. Sistem ini memanfaatkan teknologi komunikasi MQTT yang ringan dan efisien untuk perangkat IoT, serta memudahkan pemantauan lokasi dari jarak jauh melalui jaringan internet. Meski sistem berjalan sesuai fungsinya, terdapat potensi gangguan akurasi pada data GPS yang harus diantisipasi agar informasi yang diterima benar-benar valid. Dengan pengolahan data dan validasi tambahan, sistem ini dapat diandalkan untuk pelacakan lokasi bus kampus atau halte digital.

Kesimpulan dari analisa juga menunjukkan bahwa kesalahan koordinat dapat terjadi karena keterbatasan modul GPS atau lingkungan operasional, namun dapat diatasi melalui kombinasi solusi teknis seperti validasi data, penempatan modul di area terbuka, serta optimasi pengambilan data di sisi perangkat lunak. Dengan demikian, sistem dapat ditingkatkan ketepatannya dan menjadi lebih stabil dalam pemakaian jangka panjang.

3.2 Saran

Untuk pengembangan ke depan, sistem tracer bus ini dapat ditingkatkan dari sisi keandalan dan ketepatan pembacaan lokasi. Salah satu saran utama adalah menambahkan mekanisme validasi data GPS di sisi pengirim, misalnya dengan memastikan bahwa koordinat yang dikirim benar-benar valid (sudah fix) dan akurasinya memadai, agar informasi yang ditampilkan tidak menyesatkan pengguna. Selain itu, sebaiknya ditambahkan fungsi filter atau penghalusan (smoothing) data koordinat agar tampilan lokasi lebih stabil dan tidak terlalu sering berubah akibat fluktuasi kecil.

Dari sisi perangkat keras, penggunaan modul GPS dengan sensitivitas lebih tinggi atau modul dengan dukungan antena eksternal bisa dipertimbangkan untuk meningkatkan akurasi terutama saat digunakan di area dengan banyak penghalang. Pada rangkaian

penerima, akan lebih baik jika ditambahkan indikator sinyal (misalnya jumlah satelit atau kekuatan sinyal WiFi) agar pengguna dapat mengetahui kualitas koneksi secara langsung.

Untuk menunjang keandalan daya, disarankan menggunakan baterai lithium yang memiliki proteksi arus dan tegangan, atau bila digunakan di lokasi tetap, bisa langsung memanfaatkan adaptor USB 5V secara stabil tanpa baterai. Terakhir, uji coba sistem sebaiknya dilakukan dalam berbagai kondisi lingkungan dan waktu yang berbeda, agar diketahui performa sistem secara nyata sebelum diterapkan secara penuh dalam skala operasional. Jika sistem sudah stabil, pengembangan ke tampilan peta visual bisa dilakukan secara bertahap di masa depan.

LAMPIRAN

Project Kel 3 Explore CSC_Tracker BIPOL

